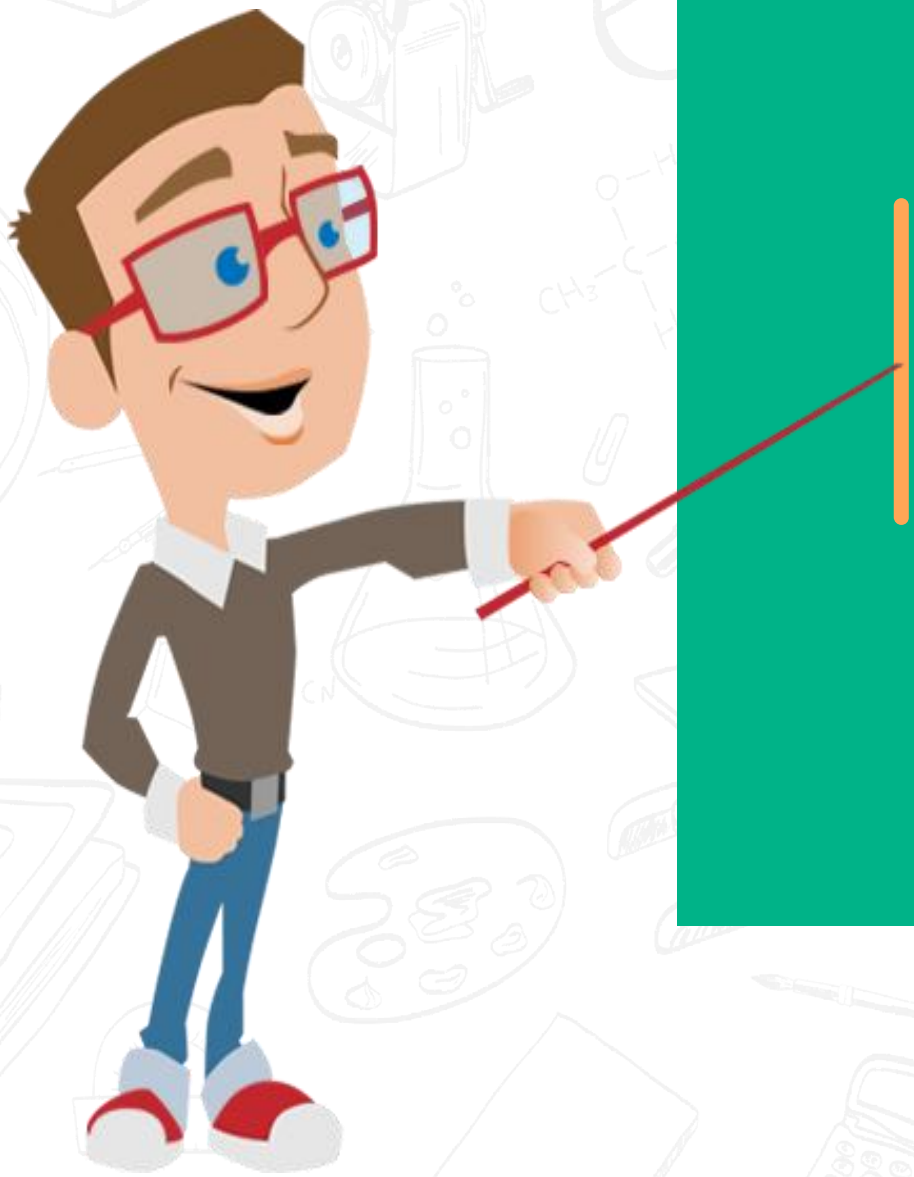


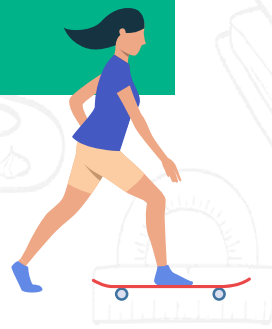
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 14-1

1. Xacro 集成 Gazebo
2. Xacro、Gazebo 和 Rviz 综合案例
3. ~
4. ~

Objectives

1、掌握 Xacro 集成 Gazebo 方法



1

Xacro 集成 Gazebo

1.1 URDF Format-Look back

1.1.1 URDF：https://wiki.ros.org/urdf/XML

- URDF 文件是一个标准的 XML 文件，在 ROS 中预定义了一系列的标签用于描述机器人模型
- URDF 的标签可以简化为两部分：连杆(link 标签) 与 关节(joint 标签)

(1) robot 根标签

(2) link 连杆标签

(3) joint 关节标签

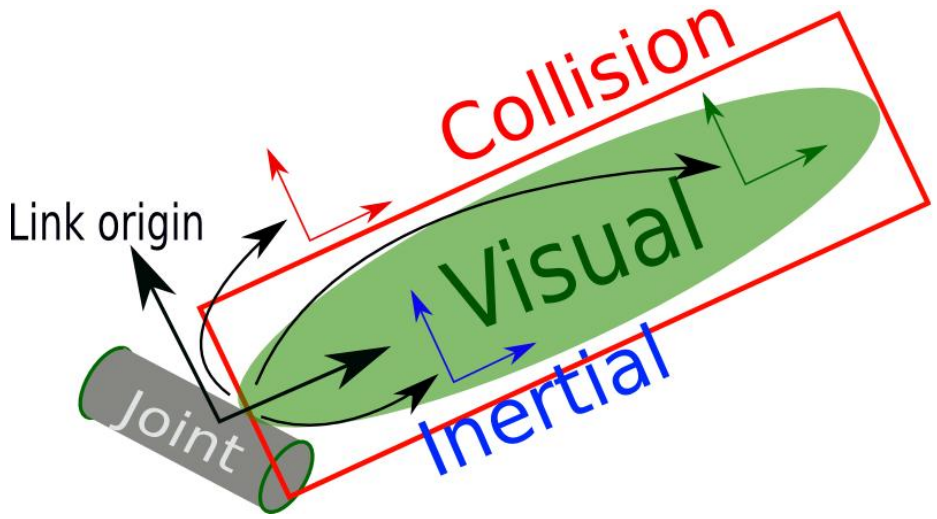
(4) gazebo 标签

1.1 URDF Format-Look back

1.1.2 Link

(2) link 标签：

- 属性
  - name=“连杆名称”
- 子标签
  - visual：描述零件外观 ☑
  - collision：描述连杆的碰撞属性
  - inertial：描述连杆的惯性矩阵



1.2 Xacro and Gazebo

1.2.1 Overview

(1) Xacro 与 Gazebo 集成的基本流程

Xacro 与 Gazebo 集成流程与 Rviz 实现类似，主要步骤如下:

1. 创建功能包(或继续使用rcdp功能包)，导入依赖项
2. 编写 URDF 或 Xacro 文件
3. 启动 Gazebo 并显示机器人模型



创建功能包(或继续使用rcdp功能包)，导入依赖项

功能包依赖项：urdf、xacro、gazebo_ros、gazebo_ros_control、gazebo_plugins

1.2 Xacro and Gazebo

1.2.1 Overview

gazebo_ros:

- 提供了ROS插件，这些插件支持消息和服务发布者，使ROS与gazebo交互
- 通过这些插件，ROS可以与gazebo进行通信，实现机器人模型的加载、控制和传感器数据的获取等功能

gazebo_ros_control:

- ROS中用于控制Gazebo仿真的插件集合。它提供了一个基于pluginlib的接口，用于在gazebo和ros_control之间实现自定义接口，以模拟更复杂的机制（非线性弹簧，连杆等）

gazebo_plugins:

- 包含了独立于机器人的gazebo插件，用于传感器。这些插件可以提供对physics::Model API、sensors::Sensor API和rendering::Visual API的访问

1.2.2 Xacro

编写 Xacro 文件

注意， 当 Xacro 与 Gazebo 集成时，和 Rviz 有以下区别:

(1) 必须使用 collision 标签，因为仿真环境涉及到碰撞检测，collision 提供碰撞检测的依据

(2) 必须使用 inertial 标签，此标签标注了当前机器人某个刚体部分的惯性矩阵，用于一些力学相关的仿真计算

(3) 必须使用 gazebo 标签，gazebo 中的颜色设置需要使用 gazebo 标签

1.2.2 Xacro

(1) collision 标签

如果机器人 link 是标准的几何体形状，collision 标签设置为和 visual 标签一致即可

(2) inertial 标签

惯性矩阵的设置需要结合 link 的质量与外形参数计算生成。标准的球体、圆柱与立方体的惯性矩阵可由公式进行计算(已经封装为 xacro 实现)

说明：除了 root_link 外，机器人的每个刚体部分都需要设置惯性矩阵，且惯性矩阵必须经计算得出，如果随意定义刚体部分的惯性矩阵，那么可能会导致机器人在 Gazebo 中出现漂移、抖动，翻滚等现象

1.2.2 Xacro

1) 球体惯性矩阵

```
<xacro:macro name="sphere_inertial_matrix" params="m r">
    <inertial>
        <mass value="${m}" />
        <inertia ixx="${2*m*r*r/5}" ixy="0" ixz="0"
            iyy="${2*m*r*r/5}" iyz="0"
            izz="${2*m*r*r/5}" />
    </inertial>
</xacro:macro>
```

1.2.2 Xacro

2) 圆柱惯性矩阵

```
<xacro:macro name="cylinder_inertial_matrix" params="m r h">
    <inertial>
        <mass value="${m}" />
        <inertia ixx="${m*(3*r*r+h*h)/12}" ixy = "0" ixz = "0"
            iyy="${m*(3*r*r+h*h)/12}" iyz = "0"
            izz="${m*r*r/2}" />
    </inertial>
</xacro:macro>
```

1.2.2 Xacro

3) 立方体惯性矩阵

```
<xacro:macro name="Box_inertial_matrix" params="m l w h">
    <inertial>
        <mass value="${m}" />
        <inertia ixx="${m*(h*h + l*l)/12}" ixy = "0" ixz = "0"
          iyy="${m*(w*w + l*l)/12}" iyz= "0"
          izz="${m*(w*w + h*h)/12}" />
    </inertial>
</xacro:macro>
```

1.2.2 Xacro

(3) 颜色设置

```
<gazebo reference="link 节点名称">
    <material>Gazebo/Blue</material>
</gazebo>
```

material 标签中，**设置的值区分大小写**，颜色可以设置为 Red Blue Green Black 等 gazebo 内置颜色

1.2.3 Gazebo

补充1：gazebo简介　参考：https://www.ngui.cc/51cto/show-28107.html?action=onClick

- 运行 Gazebo 仿真实例所涉及的两个主要组件是 gzserver 和 gzclient
- gzserver 负责解析与模拟场景以及其中的对象相关的描述文件，然后使用物理和传感器引擎模拟整个场景
- gzclient 提供图形客户端，该客户端可连接到 gzserver 并渲染仿真场景以及有用的交互工具
- 在终端中输入 gazebo 后，会先启动 gzserver，然后运行 gzclient

1.2 Xacro and Gazebo

1.2.3 Gazebo

补充1：gazebo简介

- 打开 gazebo 时出现 error：process has died 的报错，虽然仍然可以打开 gazebo，但是 gazebo 显示的并不是 launch 文件调用的 world 文件与 urdf文件内容，可能显示之前使用 gazebo 时的内容，或者出现机器人模型乱转的情况

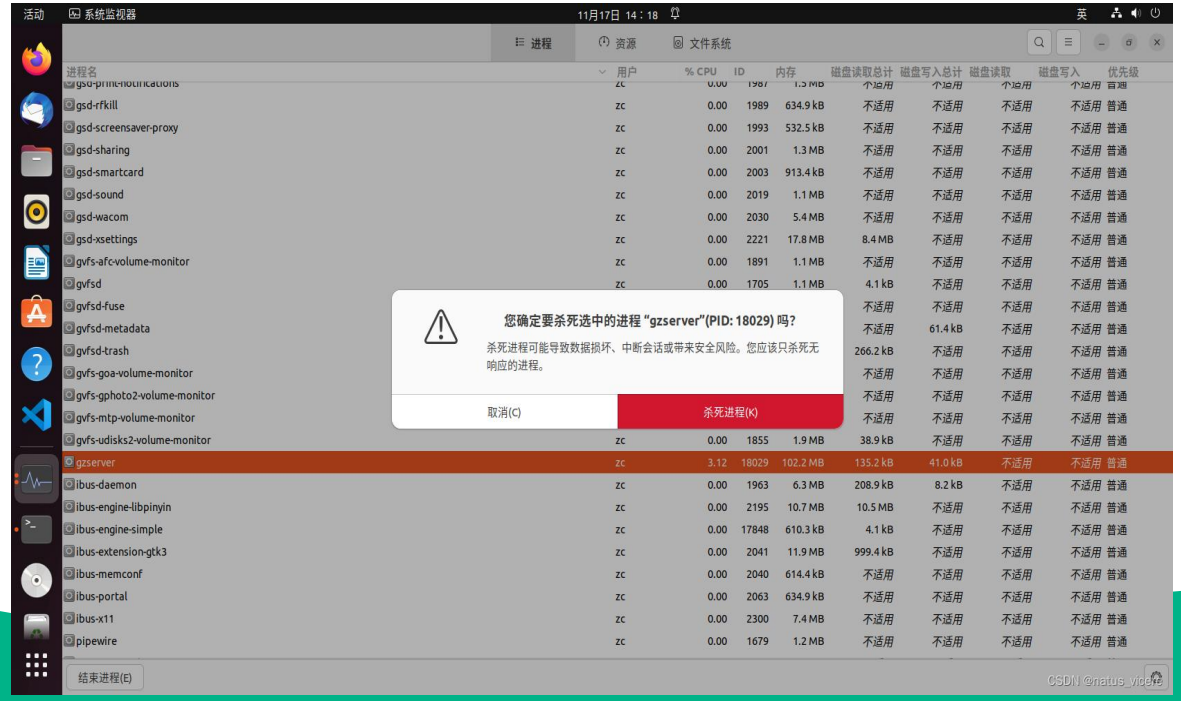
- 原因： gazebo 未完全关闭

- 可尝试的解决方案，在终端输入：

```
killall gzserver
```

```
killall gzclient
```

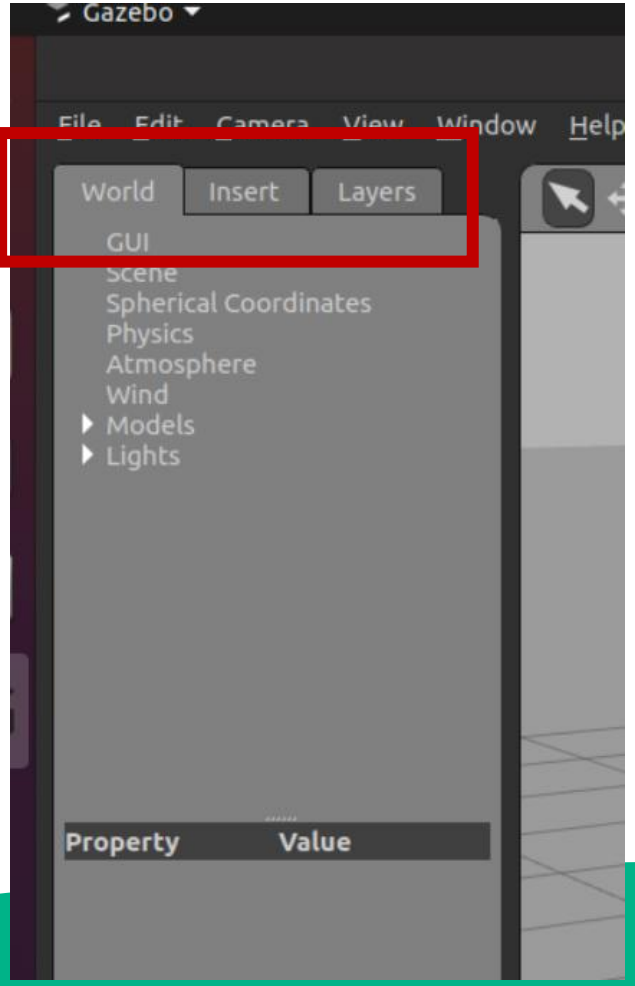
若仍无效可打开系统监视器，强行关闭 gzserver 和 gzclient



1.2 Xacro and Gazebo

1.2.3 Gazebo

补充1：gazebo简介



World选项卡：

- 显示场景中当前的灯光和模型。通过单击单个模型，可以查看或编辑其基本参数，例如位置和方向
- 可以通过“Physics”选项更改场景的物理场，例如重力和磁场。GUI选项提供对默认摄像机视角和姿势的访问

Insert选项卡：

- 显示添加到模拟场景中的模型。左键单击以展开或折叠模型列表/目录。要将对象放置在场景中，在“插入”选项卡下单击模型对象，通过在选定位置单击鼠标左键将该对象放置在场景中的指定位置

Layers选项卡：

- 此选项卡展示可选功能，在大多数情况下此选项卡为空

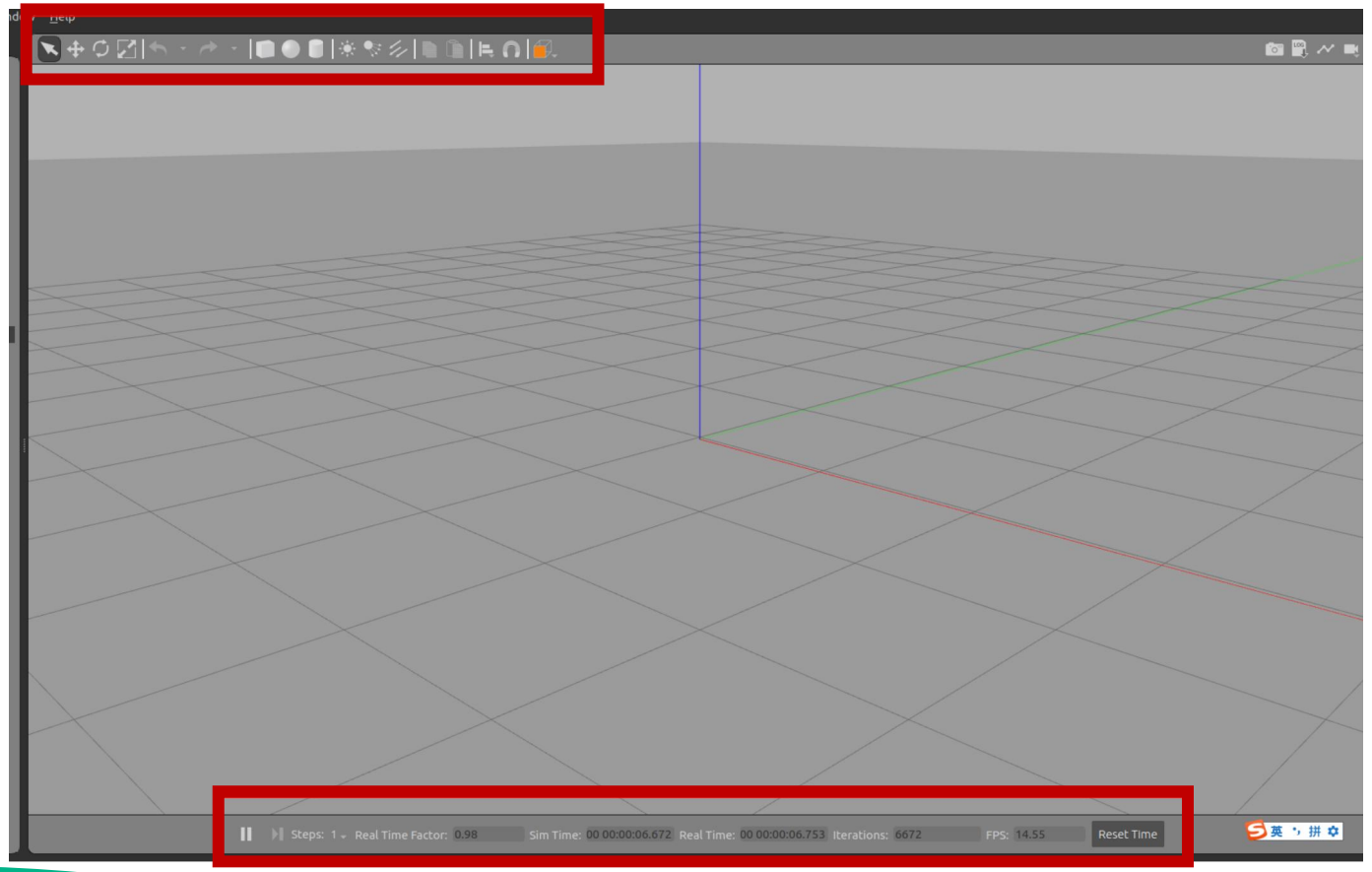
1.2 Xacro and Gazebo

1.2.3 Gazebo

补充1：gazebo简介

顶部工具栏：它可以快速访问某些光标类型，几何形状和视图

底部工具栏：底部工具栏有一个播放/暂停按钮，可以暂停模拟以方便地移动对象。该工具栏还显示有关模拟的时间数据



1.2 Xacro and Gazebo

1.2.3 Gazebo

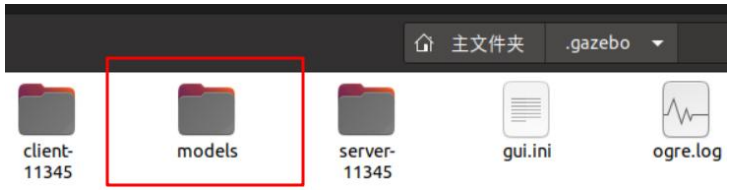
补充2：为防止 gazebo 启动黑屏或者卡顿，启动 gazebo 前在~/.gazebo/models 目录中加载模型



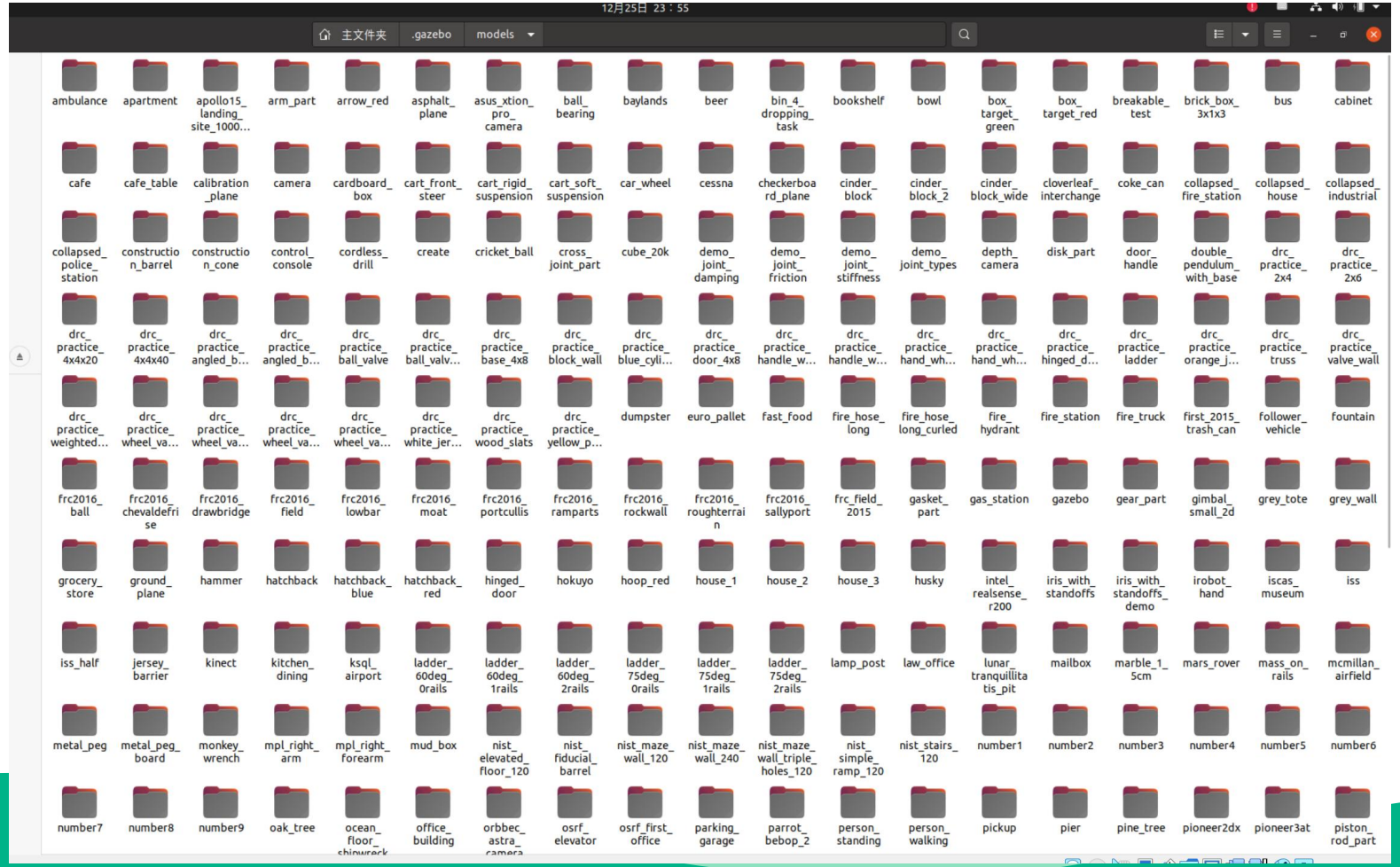
1.2 Xacro and Gazebo

1.2.3 Gazebo

补充2：为防止 gazebo 启动黑屏或者卡顿，启动 gazebo 前在~/.gazebo/models 目录中加载模型



若 .gazebo 目录中没有models目录，则新建models目录

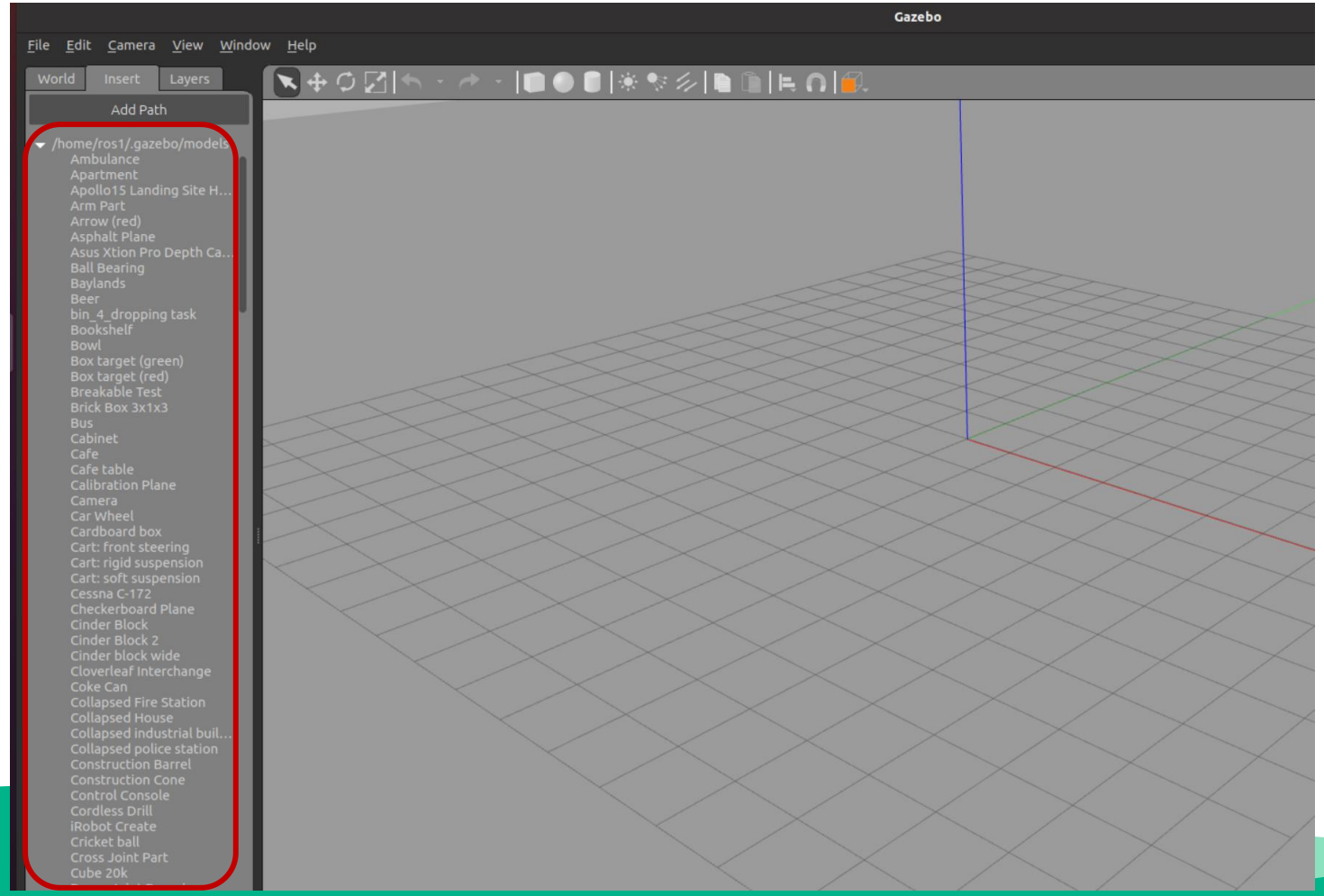


1.2 Xacro and Gazebo

1.2.3 Gazebo

补充2：为防止 gazebo 启动黑屏或者卡顿，启动 gazebo 前在~/.gazebo/models 目录中加载模型

加载的模型



1.2 Xacro and Gazebo

1.2.4 launch文件

myrobot_gazebo.launch：

```
<launch>
  <param name="robot_description" command="$(find xacro)/xacro $(find
rcdp)/urdf/gazebo/myrobot_gazebo.xacro" />

  <include file="$(find gazebo_ros)/launch/empty_world.launch" >
    <arg name="world_name" value="$(find rcdp)/worlds/box_house.world" />
  </include>

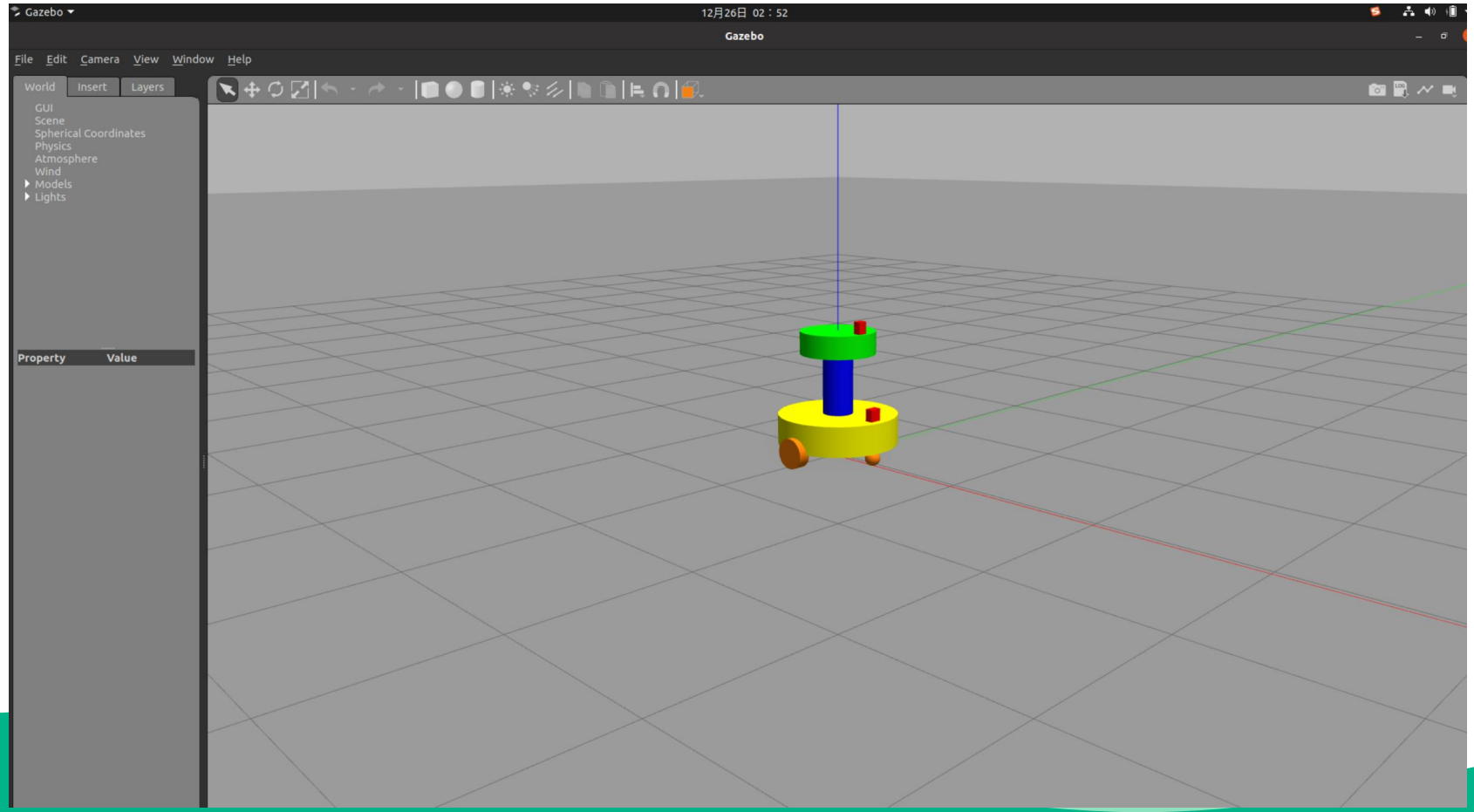
  <node pkg="gazebo_ros" type="spawn_model" name="model" args="-urdf -model
myrobot_gazebo -param robot_description"  />
</launch>
```

1.2 Xacro and Gazebo

1.2.5 launch文件解析

```
<include file="$(find gazebo_ros)/launch/empty_world.launch" >
</include>
```

会启动 gazebo 提供的空白世界虚拟环境

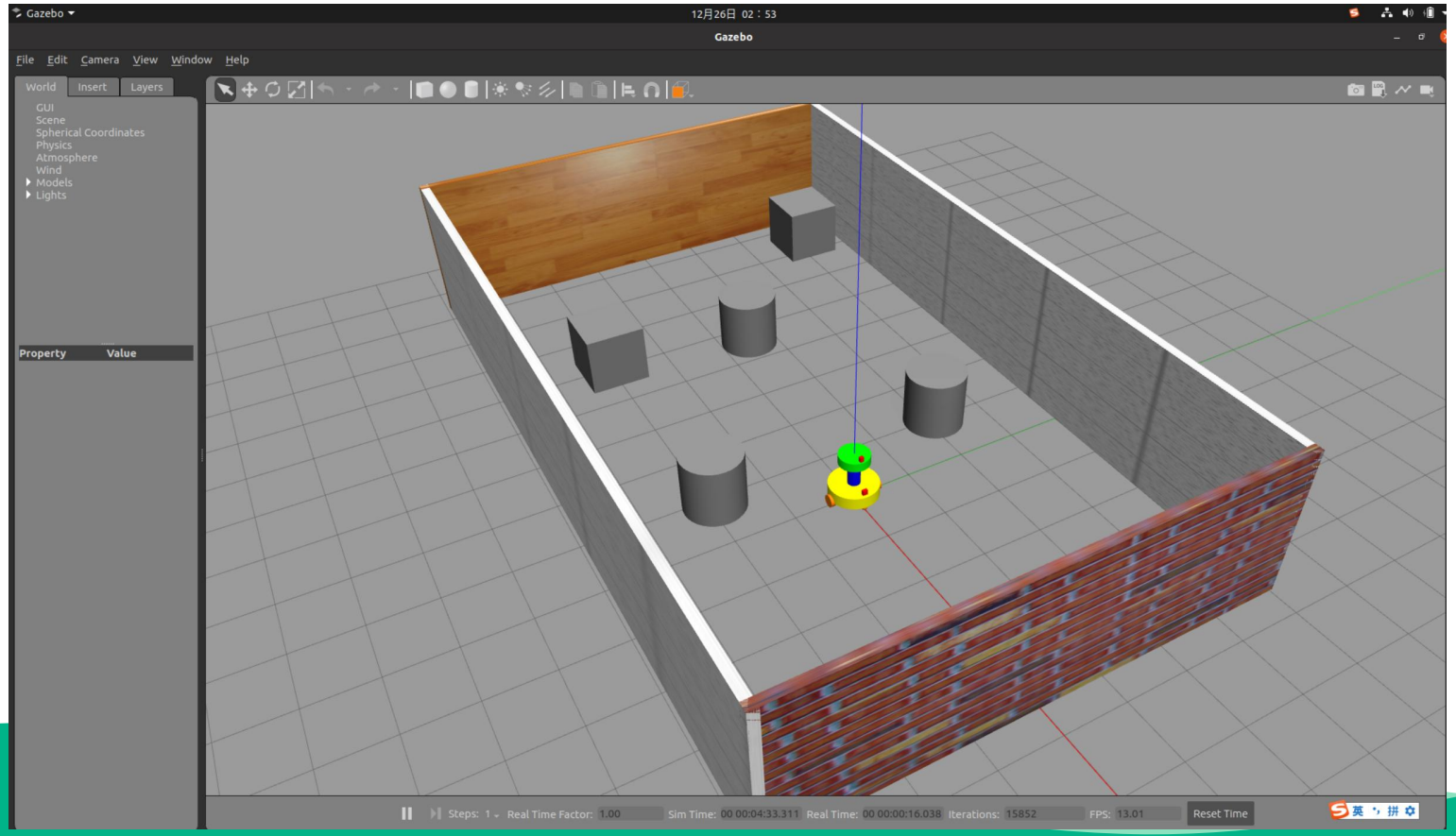


1.2 Xacro and Gazebo

1.2.5 launch文件解析

```
<include file="$(find gazebo_ros)/launch/empty_world.launch" >
    <arg name="world_name" value="$(find rcdp)/worlds/box_house.world" />
</include>
```

会启动 rcdp 功能包中 worlds 目录中的 box_house.world (自建虚拟环境)



1.2 Xacro and Gazebo

1.2.5 launch文件解析

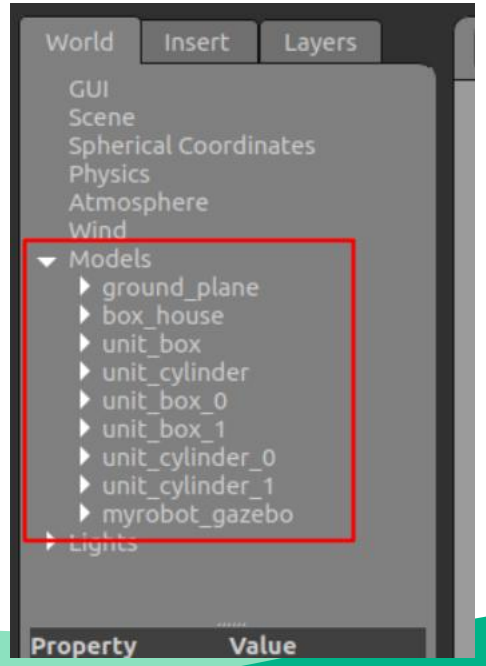
```
<node pkg="gazebo_ros" type="spawn_model" name="model" args="-
urdf -model myrobot_gazebo -param robot_description"  />
```

- 在 Gazebo 中加载一个机器人模型，该功能由 gazebo_ros 下的 spawn_model 提供：

  -urdf 加载的是 URDF 文件

  -model myrobot_gazebo 模型名称是 myrobot_gazebo

  -param robot_description 从参数 robot_description 中载入模型



1.2.6 Gazebo仿真环境搭建

Gazebo 中创建仿真实现方式有两种:

方式1: 直接添加内置组件创建仿真环境

方式2: 手动绘制仿真环境(更为灵活)

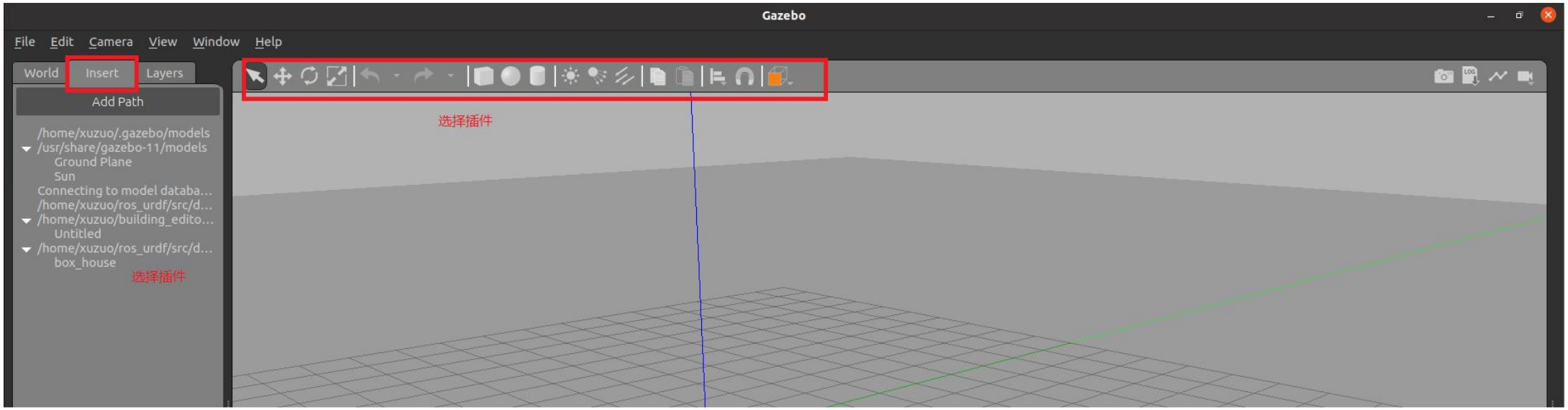
还可以直接下载使用官方或第三方提高的仿真环境插件

1.2 Xacro and Gazebo

1.2.6 Gazebo仿真环境搭建

1. 添加内置组件创建仿真环境：

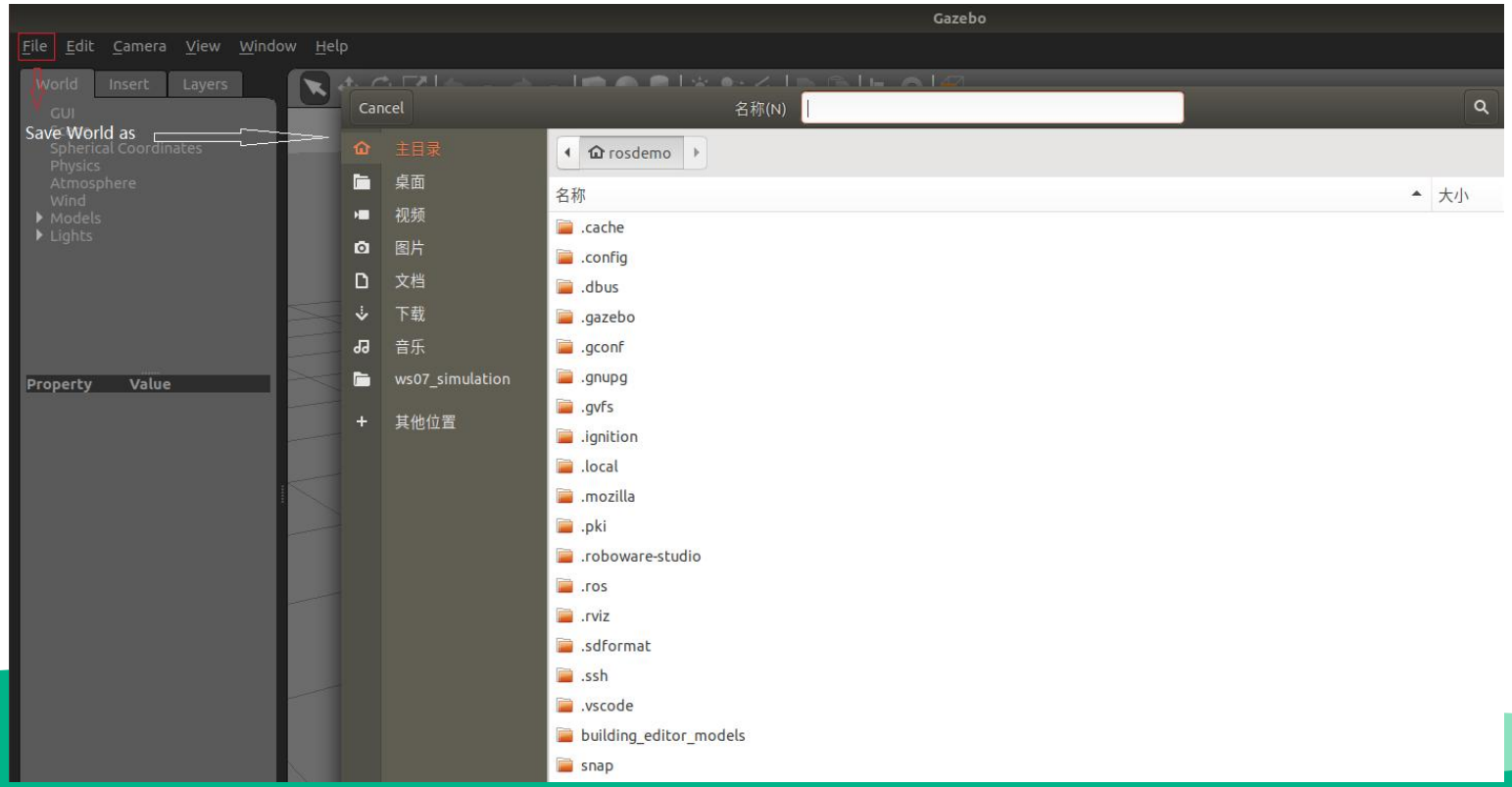
1.1 启动 Gazebo 并添加组件



1.2.6 Gazebo仿真环境搭建

1.2 保存仿真环境：

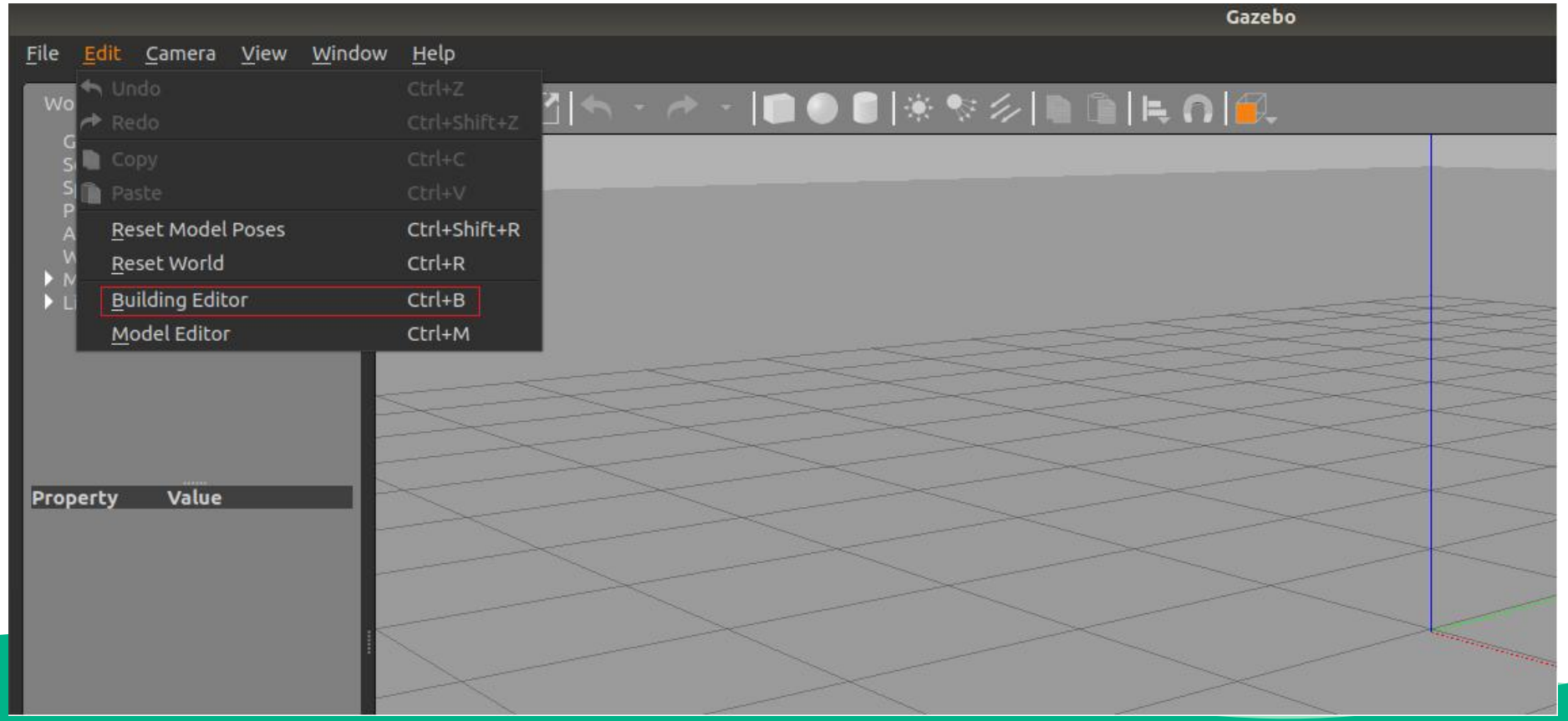
添加完毕后，选择 file ---> Save World as 选择保存路径(rcdp 功能包下的 worlds 目录)，文件名自定义，后缀名设置为 .world



1.2.6 Gazebo仿真环境搭建

2. 自定义仿真环境

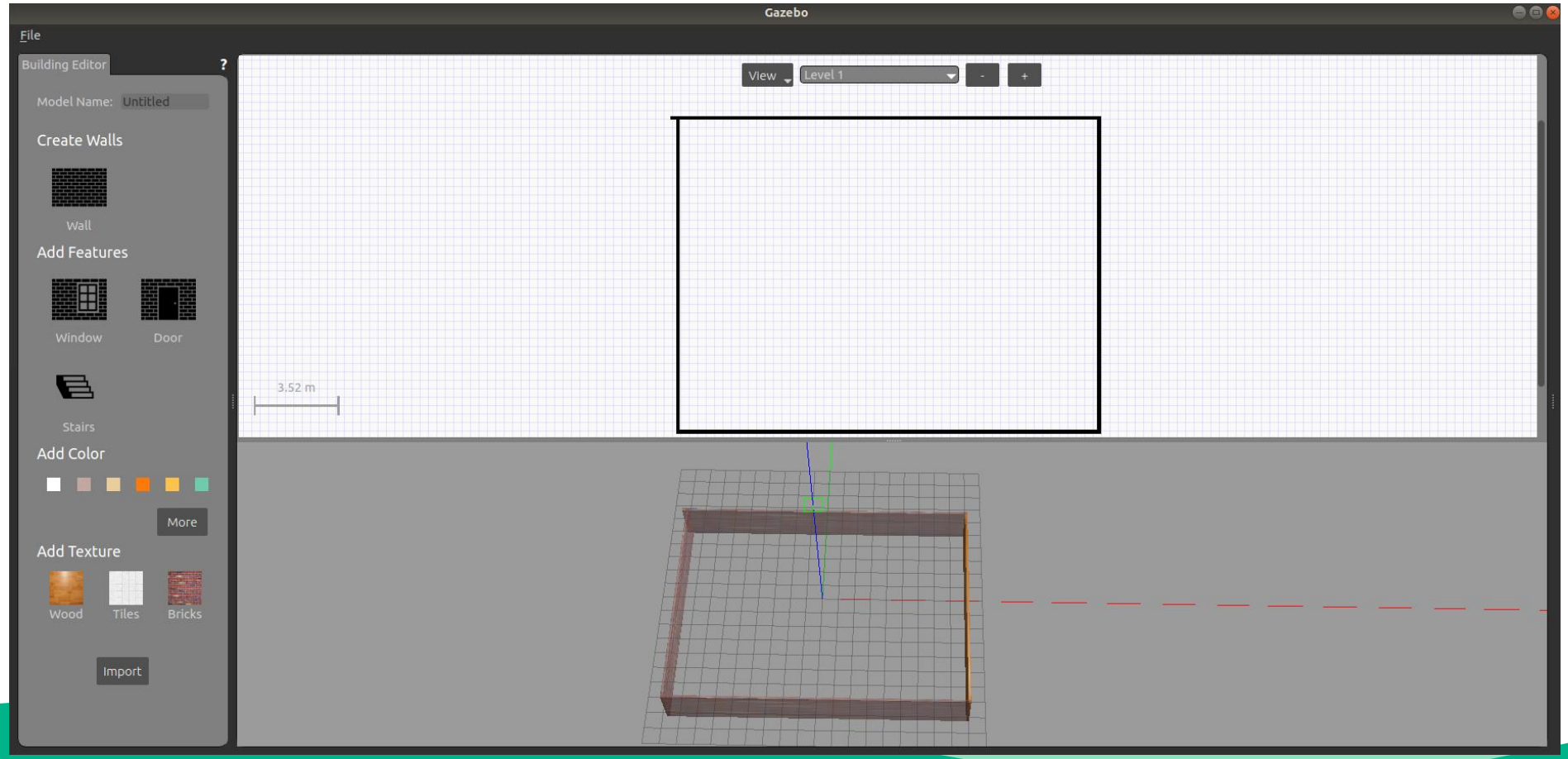
2.1 启动 gazebo 打开构建面板，绘制仿真环境



1.2 Xacro and Gazebo

1.2.6 Gazebo仿真环境搭建

2.1 启动 gazebo 打开构建面板，绘制仿真环境



1.2.6 Gazebo仿真环境搭建

2.2 保存构建的仿真场景

点击: 左上角 file ---> Save (保存路径：rcdp 功能包下 worlds 目录的 models 目录)

然后 file ---> Exit Building Editor

2.3 保存为 world 文件

像方式1一样添加一些插件，然后保存为 world 文件(保存路径：rcdp 功能包下 worlds 目录)

3. 使用官方提供的插件

git clone https://github.com/osrf/gazebo_models

2

Xacro、Gazebo 和 Rviz 综合案例

2.1.1 机器人运动控制

1. ros_control 简介

- 问题：同一套 ROS 程序，如何部署在不同的机器人系统上，如何保证 ROS 程序的可移植性？ROS 内置的解决方式是 ros_control
- ros_control 是一组软件包，它包含了控制器接口，控制器管理器，传输和硬件接口。ros_control 是一套机器人控制的中间件，是一套规范，不同的机器人平台只要按照这套规范实现，那么就可以保证与 ROS 程序兼容。通过这套规范，实现了一种可插拔的架构设计，大大提高了程序设计的效率与灵活性
- gazebo 已经实现了 ros_control 的相关接口，如果需要在 gazebo 中控制机器人运动，直接调用相关接口即可

2.1.1 机器人运动控制

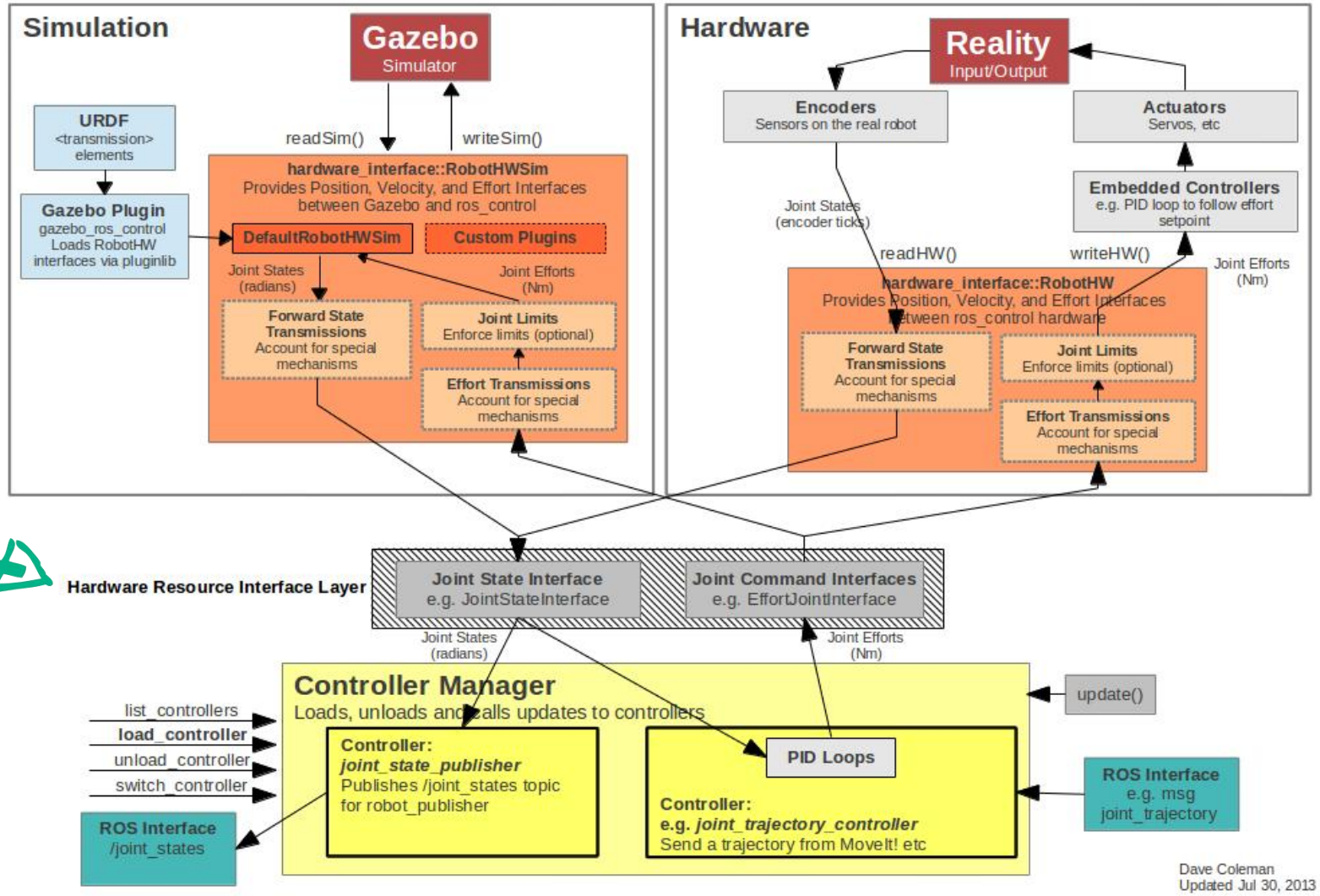
2. 运动控制实现流程(Gazebo)

- 运动控制基本流程:

1. 已经创建完毕的机器人模型，编写一个单独的 Xacro 文件，为机器人模型添加传动装置以及控制器
2. 将此文件集成到 Xacro 文件
3. 启动 Gazebo 并发布 /cmd_vel 消息控制机器人运动

2.1 Xacro、Gazebo and Rviz

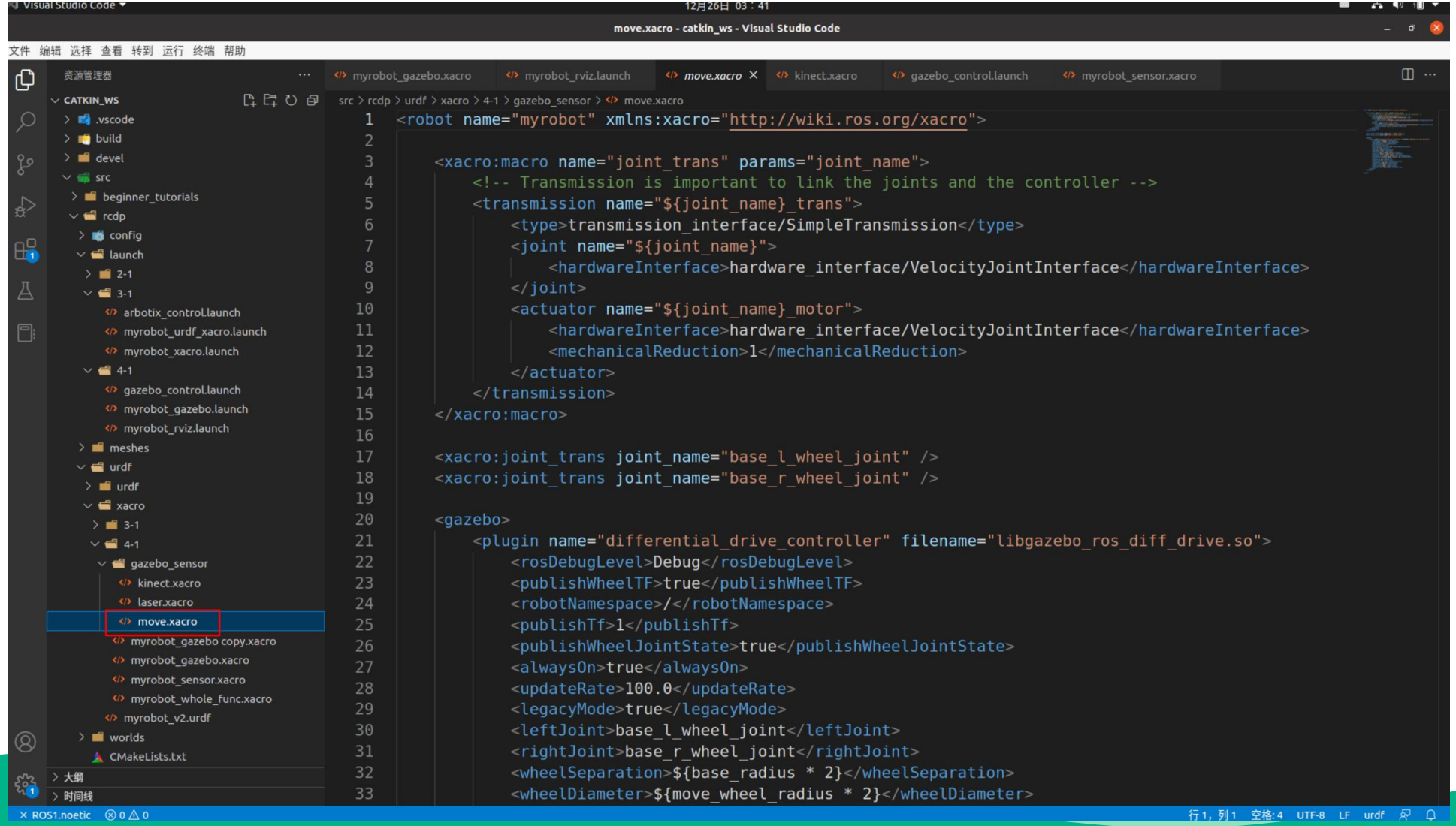
2.1.1 机器人运动控制



传动装置以及控制器

2.1 Xacro、Gazebo and Rviz

2.1.1 机器人运动控制



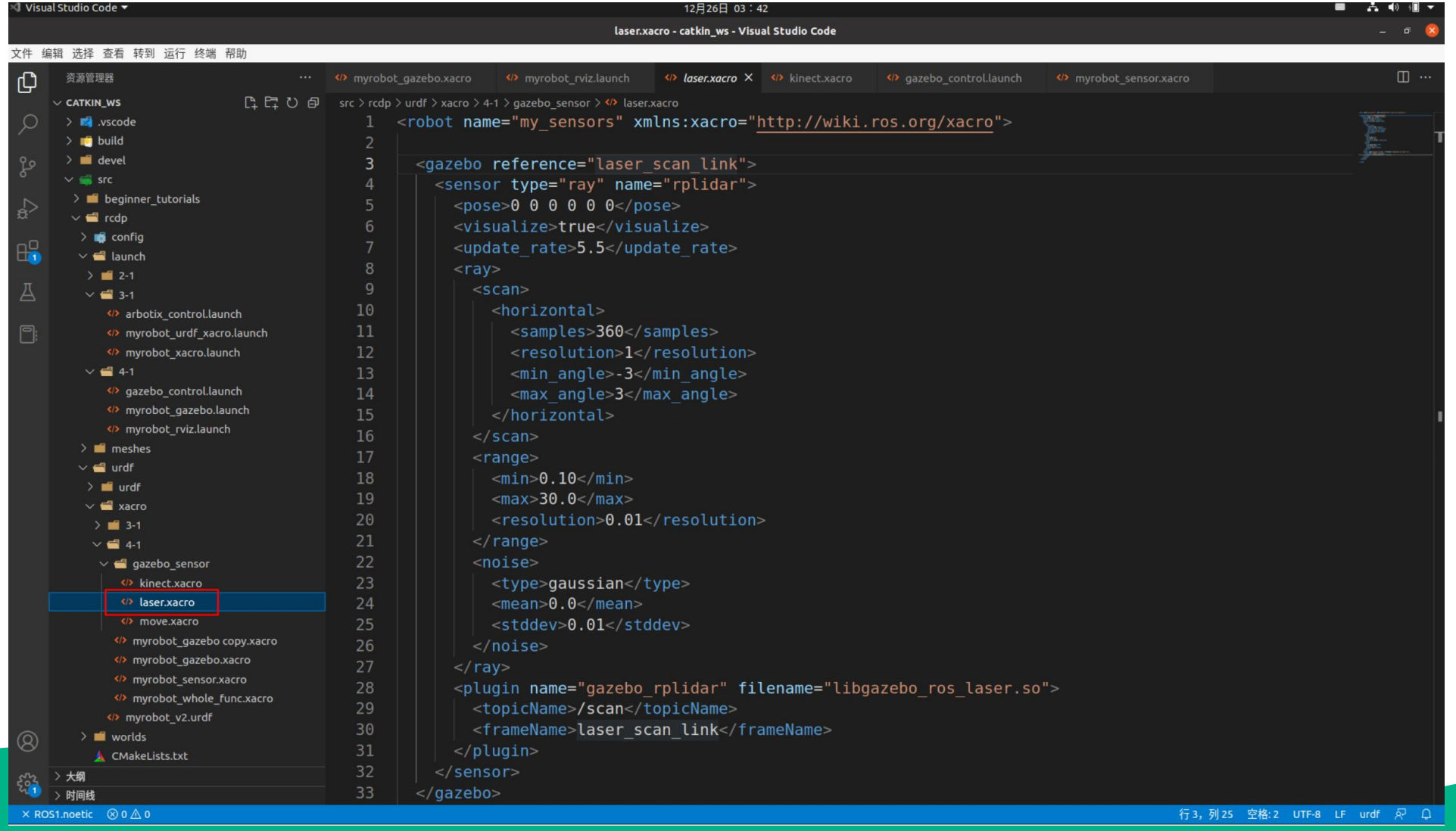
2.1.2 雷达信息仿真及显示

- 雷达仿真基本流程

1. 已经创建完毕的机器人模型，编写一个单独的 xacro 文件，为机器人模型添加雷达配置
2. 将此文件集成进 xacro 文件
3. 启动 Gazebo，使用 Rviz 显示雷达信息

2.1 Xacro、Gazebo and Rviz

2.1.2 雷达信息仿真及显示



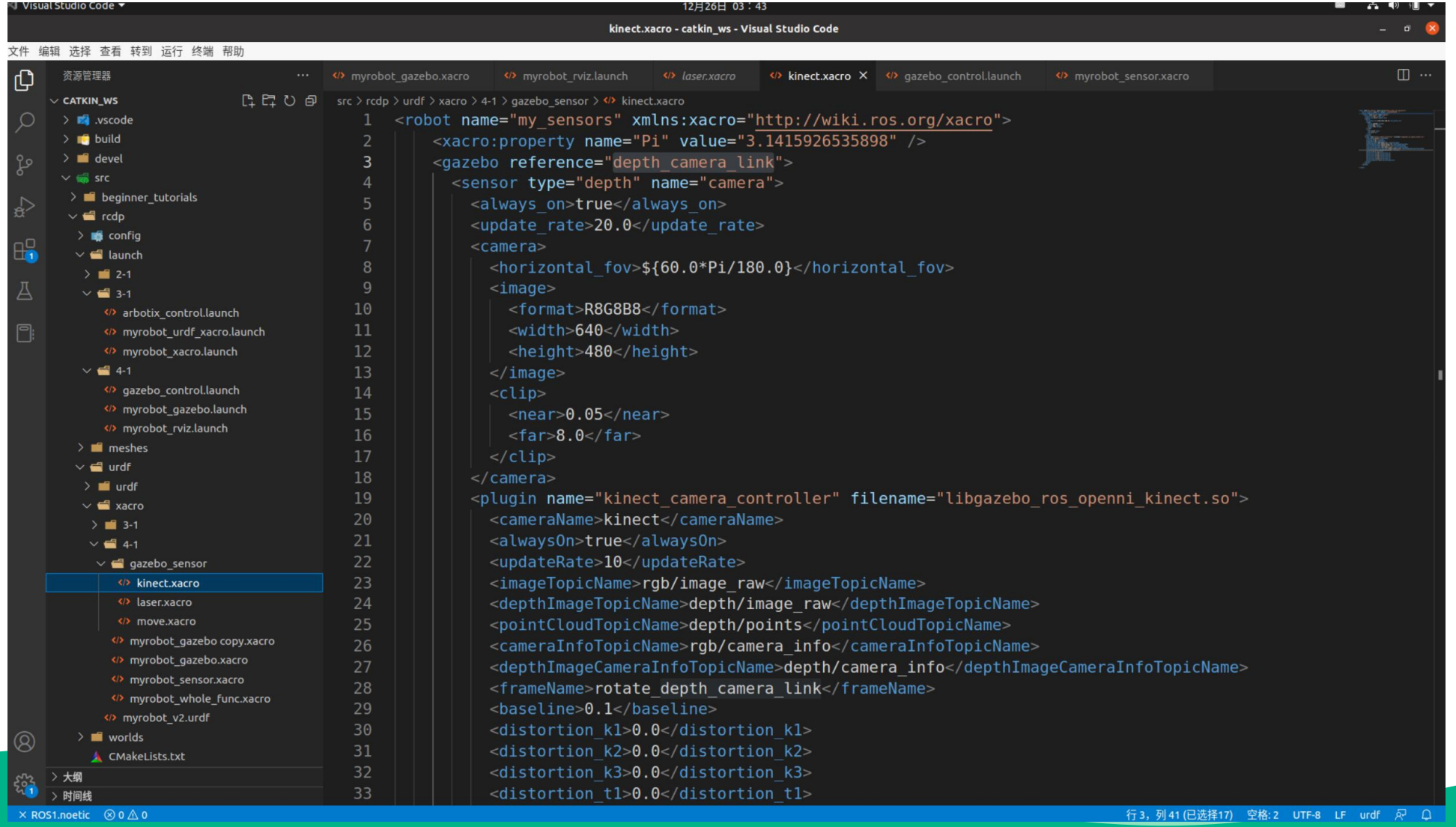
2.1.3 Kinect摄像头信息仿真及显示

- Kinect 摄像头仿真基本流程

1. 已经创建完毕的机器人模型，编写一个单独的 Xacro 文件，为机器人模型添加 Kinect 配置
2. 将此文件集成进 xacro 文件
3. 启动 Gazebo，使用 Rviz 显示 Kinect 摄像头信息

2.1 Xacro、Gazebo and Rviz

2.1.3 Kinect摄像头信息仿真及显示



2.1.4 launch文件集成-gazebo

myrobot_sensor.xacro:

```
<robot name="myrobot" xmlns:xacro="http://wiki.ros.org/xacro">
    <xacro:include filename="myrobot_gazebo.xacro" />
    <xacro:include filename="gazebo_sensor/move.xacro" />
    <xacro:include filename="gazebo_sensor/laser.xacro" />
    <xacro:include filename="gazebo_sensor/kinect.xacro" />
</robot>
```

2.1.4 launch文件集成-gazebo

gazebo_control.launch：

```
<launch>
1   <param name="robot_description" command="$(find xacro)/xacro $(find
rcdp)/urdf/gazebo/myrobot_sensor.xacro" />
2   <include file="$(find gazebo_ros)/launch/empty_world.launch" >
      <arg name="world_name" value="$(find rcdp)/worlds/box_house.world" />
    </include>
3   <node pkg="gazebo_ros" type="spawn_model" name="model" args="-urdf -model
myrobot -param robot_description"  />
4   <node pkg="teleop_twist_keyboard" type="teleop_twist_keyboard.py"
name="teleop_twist_keyboard" output="screen"/>
</launch>
```

2.1.4 launch文件集成-rviz

myrobot_rviz.launch

```
<launch>
1   <node pkg="tf2_ros" type="static_transform_publisher" name="static_transform_publisher"
args="0 0 0 -1.57 0 -1.57 /depth_camera_link /rotate_depth_camera_link" />

2   <param name="robot_description" command="$(find xacro)/xacro $(find
rcdp)/urdf/gazebo/myrobot_gazebo.xacro" />

3   <node pkg="rviz" type="rviz" name="rviz" args="-d $(find rcdp)/config/myrobot_gazebo.rviz" />

4   <node pkg="joint_state_publisher" type="joint_state_publisher" name="joint_state_publisher" />

5   <node pkg="robot_state_publisher" type="robot_state_publisher" name="robot_state_publisher" />
</launch>
```

2.1.4 launch文件集成-rviz

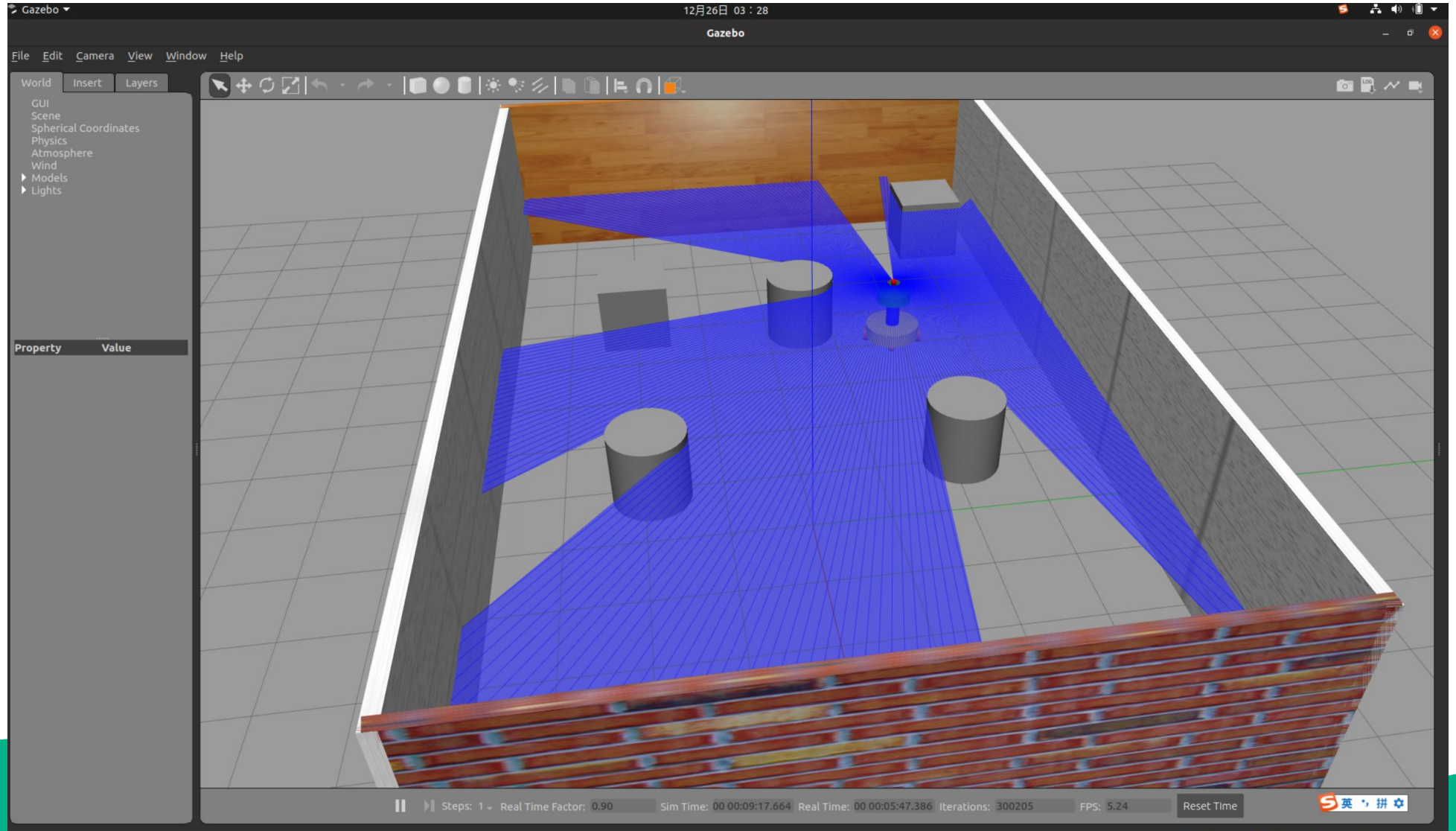
打开两个终端，按顺序启动：

```
roslaunch rcdp gazebo_control.launch
```

```
roslaunch rcdp myrobot_rviz.launch
```

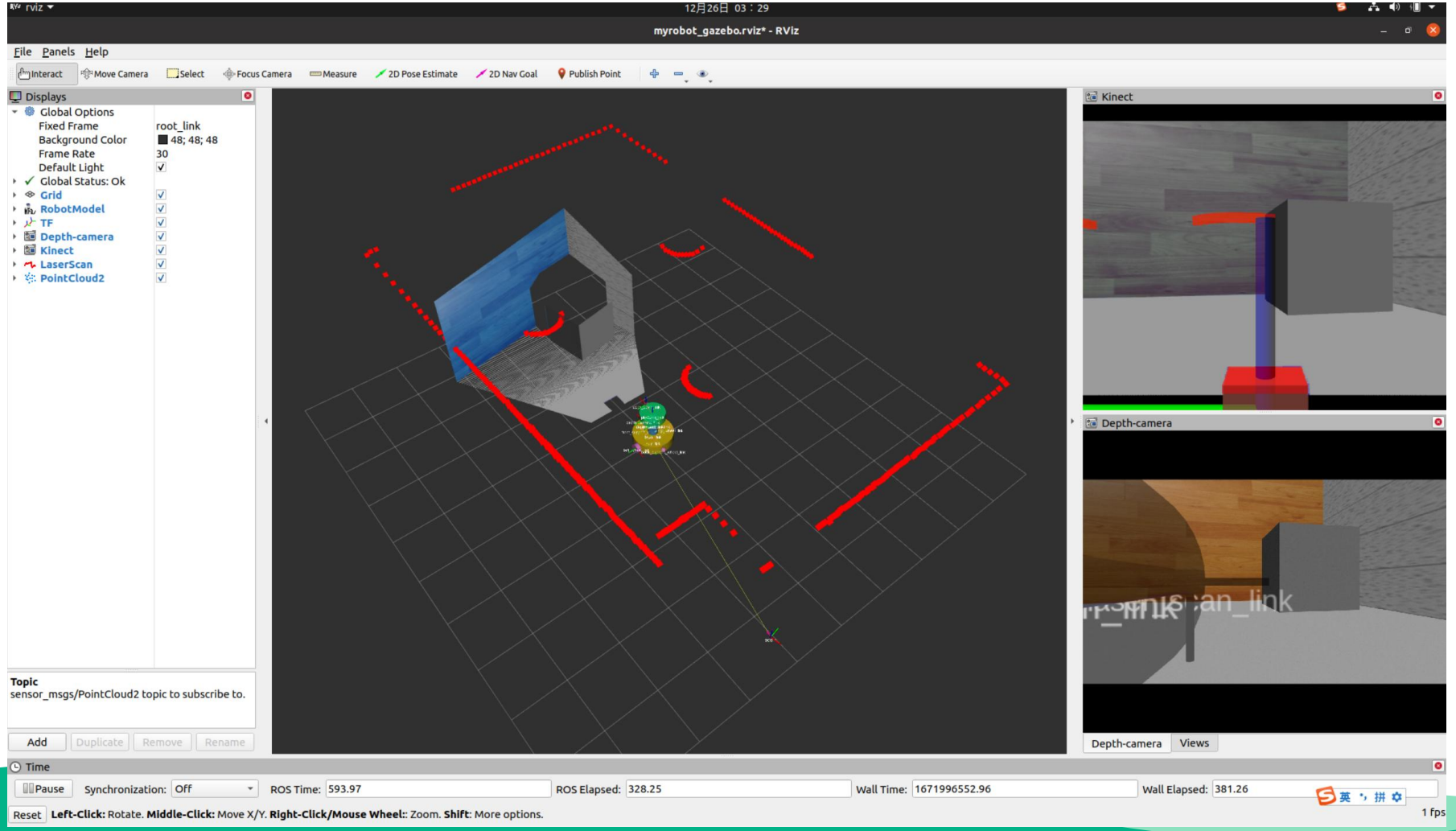
2.1 Xacro、Gazebo and Rviz

2.1.5 运行结果-gazebo



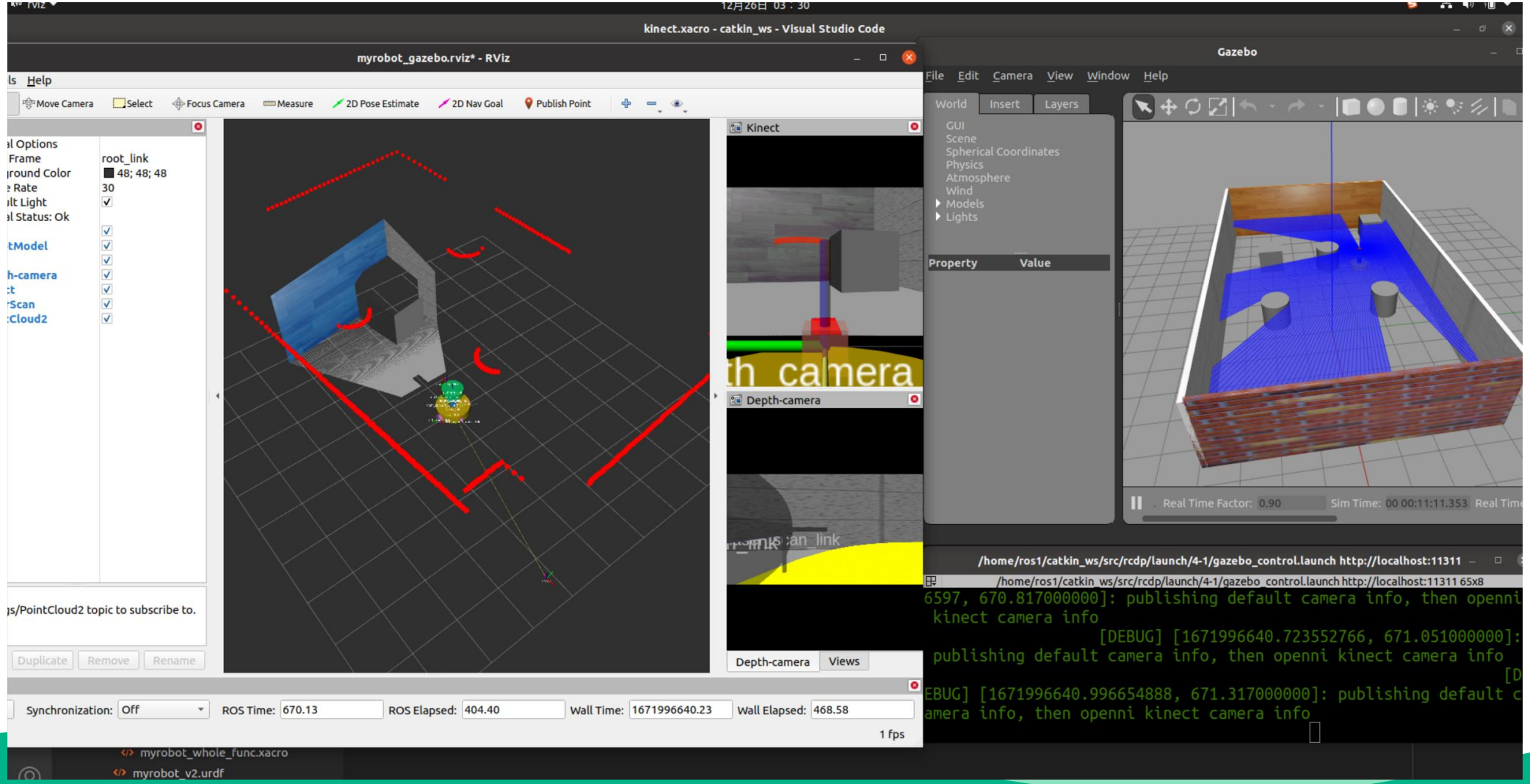
2.1 Xacro、Gazebo and Rviz

2.1.5 运行结果-rviz



2.1 Xacro、Gazebo and Rviz

2.1.5 运行结果-gazebo + rviz



Summary

1. Xacro 集成 Gazebo
    a) URDF Format-Look back
    b) Xacro and Gazebo
        1. Overview
        2. Xacro
        3. Gazebo
        4. launch文件
        5. launch文件解析
        6. Gazebo仿真环境搭建
2. Xacro、Gazebo 和 Rviz 综合案例
    a) 机器人运动控制
    b) 雷达信息仿真及显示
    c) Kinect摄像头信息仿真及显示
    d) launch文件集成
    e) 运行结果

Q&A

今日事今日毕

Question?